АРХИТЕКТУРА ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ (ROS 2)

Робот-курьер НТЦ АО «АВТОВАЗ», шасси 4WIS/4WID с крабовым ходом. Целевая платформа: **Ubuntu 24.04 / 26.04 LTS**, **ROS 2 Humble/Jazzy**.

⚠ Протокол моста

Мост `module_bridge_node` в `ros2_ws/` собран под **бинарный** протокол 10/16 байт, которого в железе нет. Реальная прошивка (`firmware/motor_wheel`, **fw26**) общается **текстовыми строками** по USB на каждое колесо. Актуальный протокол — `SERIAL_PROTOCOL.md`; §10 ниже показывает, что именно нужно заменить в `module_link.py`, чтобы ROS 2-стек заработал с настоящими платами.

На роботе этот стек не запущен. Работает бортовое ПО Zapret: своя карта занятости по лидару LDS-01, свой планировщик маршрута, без Nav2. Из ROS 2 там используется только драйвер лидара, публикующий `/scan`. Распознавания знаков и светофора в рабочем ПО нет. Документ ниже описывает справочную архитектуру пакетов `ros2_ws/`; что уточнено по Zapret — `ZAPRET_SYNC.md`.

---

1. Уровни управления

```
┌──────────────────────────────────────────────────────────────────────┐
│ УРОВЕНЬ 4. МИССИЯ И БЕЗОПАСНОСТЬ                                     │
│ rus_slam_safety: mission_manager_node (FSM миссии, доставка по точкам),
│                  safety_node (гейт скоростей, e-stop, watchdog, АКБ) │
├──────────────────────────────────────────────────────────────────────┤
│ УРОВЕНЬ 3. НАВИГАЦИЯ И ВОСПРИЯТИЕ                                    │
│ Nav2 (планирование, recovery), slam_toolbox (SLAM), AMCL (локализация),
│ rus_slam_perception: camera_node, sign_detector_node, traffic_light_node
│ rus_slam_lidar: lds01_driver_node (Лидар LDS-01 -> LaserScan)        │
├──────────────────────────────────────────────────────────────────────┤
│ УРОВЕНЬ 2. УПРАВЛЕНИЕ ДВИЖЕНИЕМ                                     │
│ rus_slam_base: crab_drive_node (обратная кинематика 4WIS/4WID),      │
│                odometry_node (одометрия + TF + JointState)           │
├──────────────────────────────────────────────────────────────────────┤
│ УРОВЕНЬ 1. МОСТ ОБЩЕНИЯ С ЖЕЛЕЗОМ (требует доработки)               │
│ rus_slam_base: module_bridge_node (4× USB 115200, текстовые кадры, CRC16)
├──────────────────────────────────────────────────────────────────────┤
│ УРОВЕНЬ 0. МИКРОКОНТРОЛЛЕРЫ (5×)                                    │
│ firmware/motor_wheel (fw26), 4 платы: Mega 2560 + TB6600 + ZS-Z11H,  │
│ текстовый протокол @F/@TLM по USB, один кабель на колесо;            │
│ firmware/sensor_module, 1 плата: свет, замок отсека, карты, GY-87    │
└──────────────────────────────────────────────────────────────────────┘
```

---

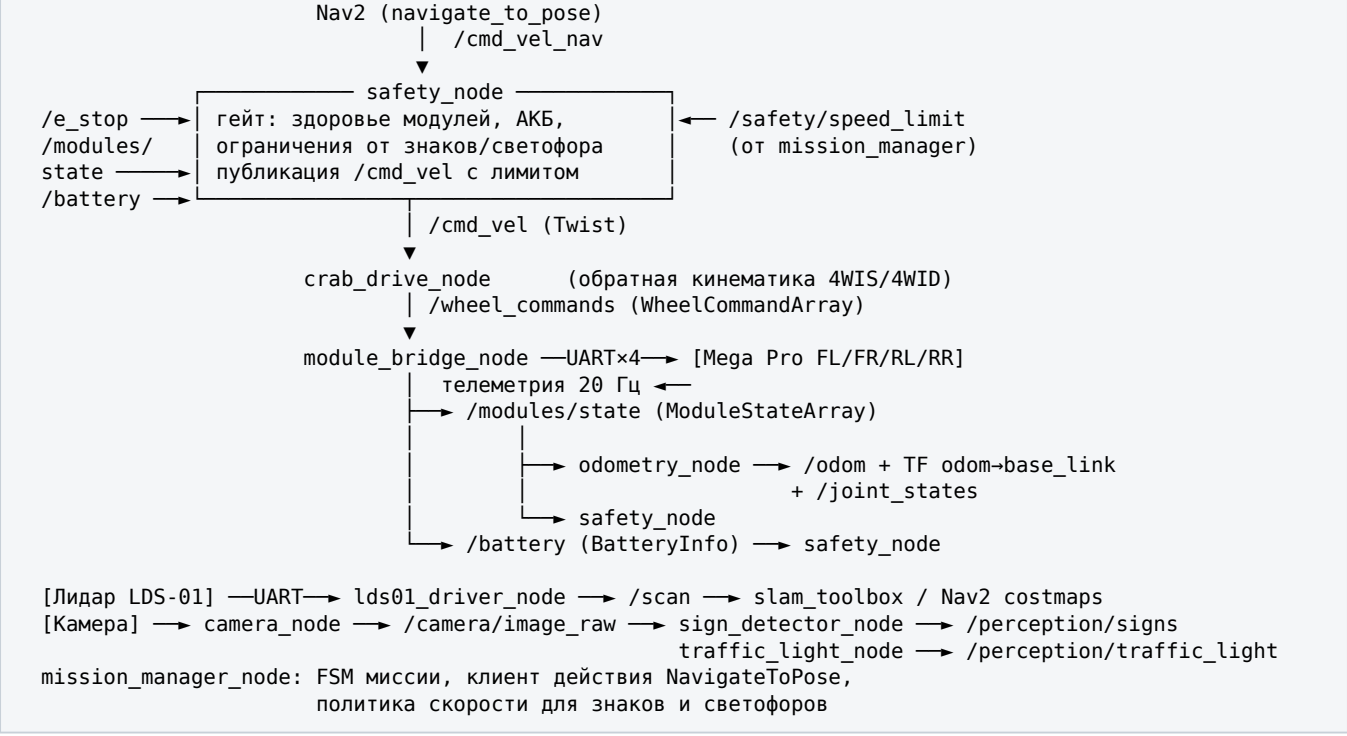
2. Состав пакетов (колкон-воркспейс `ros2_ws/src`)

Пакет	Тип	Назначение
`rus_slam_interfaces`	ament_cmake (rosidl)	Пользовательские сообщения/сервисы: команды колес, состояние модулей, АКБ, знаки, светофор, хоминг
`rus_slam_base`	ament_python	Ядро: протокол обмена, кинематика, мост UART, одометрия, TF
`rus_slam_description`	ament_python	URDF/Xacro модель робота + `robot_state_publisher`
`rus_slam_lidar`	ament_python	Драйвер Лидара LDS-01 (UART -> `sensor_msgs/LaserScan`)
`rus_slam_perception`	ament_python	Камера, детекция знаков («Стоп», «Пешеходный переход», «Искусственная неровность») и светофоров
`rus_slam_navigation`	ament_python	Конфиги и лаунчеры: slam_toolbox, Nav2, AMCL
`rus_slam_safety`	ament_python	Безопасность (гейт `/cmd_vel`, вотчдог, АКБ) и менеджер миссии (FSM)
`rus_slam_bringup`	ament_python	Точка входа: все лаунч-файлы, конфигурация, udev-правила

Зависимости сборки: `colcon build --symlink-install` в каталоге `ros2_ws`. Внешние зависимости: `python3-serial`, `python3-opencv`, `slam_toolbox`, `nav2_bringup`, `robot_state_publisher`, `xacro`. Установка одним списком:

```
sudo apt install -y python3-serial python3-opencv \
     ros-$ROS_DISTRO-slam-toolbox ros-$ROS_DISTRO-navigation2 \
     ros-$ROS_DISTRO-nav2-bringup ros-$ROS_DISTRO-robot-state-publisher \
     ros-$ROS_DISTRO-xacro ros-$ROS_DISTRO-tf-transformations
```

3. Граф узлов и топиков



3.1. Таблица топиков

Топик	Тип	Издатель	Подписчики	Частота
`/cmd_vel_nav`	`geometry_msgs/Twist`	Nav2	safety_node	до 20 Гц
`/cmd_vel`	`geometry_msgs/Twist`	safety_node	crab_drive_node	20 Гц
`/wheel_commands`	`rus_slam_interfaces/WheelCommandArray`	crab_drive_node	module_bridge_node	до 50 Гц
`/modules/state`	`rus_slam_interfaces/ModuleStateArray`	module_bridge_node	odometry_node, safety_node	20 Гц
`/battery`	`rus_slam_interfaces/BatteryInfo`	module_bridge_node	safety_node	1 Гц
`/odom`	`nav_msgs/Odometry`	odometry_node	Nav2, slam_toolbox	20 Гц
`/joint_states`	`sensor_msgs/JointState`	odometry_node	robot_state_publisher	20 Гц
`/scan`	`sensor_msgs/LaserScan`	lds01_driver_node	SLAM, costmaps	5–10 Гц
`/camera/image_raw`	`sensor_msgs/Image` (rgb8)	camera_node	детекторы	15–30 Гц
`/camera/camera_info`	`sensor_msgs/CameraInfo`	camera_node	—	15–30 Гц
`/perception/signs`	`rus_slam_interfaces/SignDetectionArray`	sign_detector_node	mission_manager	5–15 Гц
`/perception/traffic_light`	`rus_slam_interfaces/TrafficLightState`	traffic_light_node	mission_manager	5–15 Гц
`/e_stop`	`std_msgs/Bool`	внешний источник (кнопка/пульт)	safety_node	событийно
`/safety/status`	`std_msgs/String`	safety_node	mission_manager	2 Гц
`/safety/speed_limit`	`std_msgs/Float32`	mission_manager	safety_node	событийно
`/tf`, `/tf_static`	`tf2_msgs/TFMessage`	все	все	—

3.2. Сервисы и действия

Имя	Тип	Поставщик	Назначение
`/home_modules`	`rus_slam_interfaces/HomeModules`	module_bridge_node	Поиск нулевого азимута (концевики)
`/modules/enable`	`rus_slam_interfaces/SetModulesEnabled`	module_bridge_node	Включение/снятие момента с модулей
`/navigate_to_pose`	`nav2_msgs/action/NavigateToPose`	Nav2	Целевая точка маршрута (клиент — mission_manager)

3.3. Дерево TF

```
map ──(Nav2/slam_toolbox/AMCL)──► odom ──(odometry_node)──► base_link
                                                            ├─► base_footprint (static)
                                                            ├─► lidar_link    (static)
                                                            ├─► camera_link   (static)
                                                            ├─► module_fl_steer ─► wheel_fl (continuous)
                                                            ├─► module_fr_steer ─► wheel_fr
                                                            ├─► module_rl_steer ─► wheel_rl
                                                            └─► module_rr_steer ─► wheel_rr
```

Параметры `map→odom` дает навигационный стек; `odom→base_link` — узел одометрии по телеметрии энкодеров; поворотные рамы и колеса анимируются через `/joint_states`.

4. Ключевые алгоритмы

4.1. Обратная кинематика (rus_slam_base/kinematics.py)

Вход: (vx, vy, wz). Для каждого модуля i с координатами (x_i, y_i):

```
vx_i = vx − wz·y_i;  vy_i = vy + wz·x_i
v_i  = hypot(vx_i, vy_i);  θ_i = atan2(vy_i, vx_i)
```

Оптимизация перекладки: если |θ_i − θ_prev| > 90°, угол разворачивается на 180°, скорость инвертируется (колесо едет «задом» — быстрее, чем доворот). Нормализация: если максимум v_i превышает max_linear_speed, все скорости масштабируются с сохранением траектории.

4.2. Одометрия (rus_slam_base/odometry_core.py)

Вход: фактические углы модулей и пройденный путь каждого колеса (тики энкодера → метры). Каждое колесо дает вектор скорости w_i = v_i·[cosθ_i, sinθ_i], приложенный в точке (x_i, y_i). Скорость корпуса (vx, vy, wz) находится как решение переопределенной системы (МНК, нормальные уравнения 3×3). Это устойчиво к шуму отдельных энкодеров и корректно для крабового хода и вращения на месте.

4.3. Безопасность (rus_slam_safety/safety_node.py)

Гейт /cmd_vel останавливает робота при любом из событий:

1. Активен внешний e-stop (/e_stop = true).
2. Нет телеметрии модулей дольше comms_timeout (обрыв USB).
3. Напряжение АКБ ниже battery_critical_v (аварийно) или ниже battery_low_v (ползущий режим, ограничение 0.2).
4. Модуль сообщил флаг неисправности.

Ограничение скорости умножается на коэффициент /safety/speed_limit (0.0–1.0), задаваемый менеджером миссии (знак «Стоп», красный свет, «лежачий полицейский»).

4.4. Менеджер миссии (rus_slam_safety/mission_manager_node.py)

Конечный автомат: BOOT → HOMING → READY → DELIVER → DONE / FAULT. На этапе DELIVER последовательно отправляет точки маршрута из конфига mission.yaml через действие NavigateToPose. Реакции на восприятие: красный светофор → лимит 0 до зеленого; знак «Стоп» → полная остановка на stop_hold_sec; «Искусственная неровность» → лимит 0.3 на slowdown_hold_sec.

---

5. Протокол обмена с модулями (кратко)

Полное описание — в SERIAL_PROTOCOL.md; источник истины — firmware/motor_wheel/motor_wheel.ino.

- **Физика:** USB (CDC) на каждое колесо, 115200 8N1, по одной плате на колесо.
- **Команды ПК → МК:** текстовые строки, \n в конце. Примеры: w FL 12.5, v 40 f, t FL -8, b 123, k, ph 1, zq 17, zc 17. Роль (FL/FR/RL/RR) в строке присутствует, сервер удаляет её перед отправкой на порт своего колеса.
- **Телеметрия МК → ПК:**
  - @F — короткий полный снимок каждые ~30 мс (не чаще 20 мс), 20 полей + CRC16;
  - @TLM — подробный кадр ключ=значение каждые 200 мс + crc=;
  - @Z — ответ на zq/zc при записи сохранённого полного нуля;
  - # XX: … — сообщения для человека (журнал экрана).

- **Контроль целостности:** CRC16/Modbus (полином `0xA001`, init `0xFFFF`) по всем байтам строки до суффикса.
- **Сторожевой таймер платы:** нет команды тяги 650 мс → `drive = 0`, BRAKE включён, STOP снят.

> **Состояние `ros2_ws`.** `module_link.py` и `protocol.py` в этом репозитории всё ещё описывают бинарные кадры `0xAA 0x55` / `0xBB 0x44`, которых прошивка не выдаёт. Чтобы ROS 2-стек заработал с реальными платами, нужно заменить транспорт в `module_link.py` (см. §10) — рабочий референс парсинга и сборки строк есть в Zapret: `slam_gui/backend.py` (`MasterSource`, `parse_master_line`) и `robot_control/rus_core/protocol.py`.

6. Режимы запуска

Лаунч	Команда	Что поднимает
Только железо + кинематика	`ros2 launch rus_slam_bringup robot.launch.py`	description, мост, кинематика, одометрия, лидар, камера, детекторы, safety
Робот + навигация	`ros2 launch rus_slam_bringup full.launch.py`	всё выше + SLAM **или** локализация по карте + Nav2 + миссия
Симуляция без модулей	`ros2 launch rus_slam_bringup robot.launch.py sim:=true`	мост в режиме эмуляции модулей
Только картографирование	`ros2 launch rus_slam_navigation slam.launch.py`	slam_toolbox online_async
Навигация по готовой карте	`ros2 launch rus_slam_navigation navigation.launch.py map:=/path/to/map.yaml`	Nav2 + AMCL

Типовой порядок ввода в эксплуатацию:

1. `sim:=true` — проверка графа топиков без железа;
2. прошивка 4 плат ролью FL/FR/RL/RR (`firmware/README.md`), калибровка нуля `k` и потенциометра `q`;
3. хоминг: на плате `k` (или `w XX calib` с сервера), затем `q` для потенциометра;
4. картирование: записать карту через `slam_toolbox` + `map_saver_cli`;
5. боевой запуск: `full.launch.py use_sim_time:=false map:=...`.

---

7. Конфигурация

Все параметры вынесены в YAML-файлы пакета `rus_slam_bringup/config`:

Файл	Содержимое
`robot.yaml`	геометрия шасси, порты модулей, лимиты скоростей, параметры одометрии
`lidar.yaml`	порт/скорость лидара, параметры кадра протокола, frame_id
`perception.yaml`	устройство камеры, пороги детекторов, области интереса
`safety.yaml`	таймауты, пороги АКБ, статусы

Файл	Содержимое
`mission.yaml`	точки маршрута, время остановки у знаков
`nav2_params.yaml` (в `rus_slam_navigation/config`)	полный стек Nav2 под шасси 750×600 мм
`slam_toolbox.yaml`	online_async SLAM

---

8. Отказоустойчивость

Отказ	Реакция системы
Обрыв USB модуля	Плата сама тормозит тягу через 650 мс без команд; сервер помечает колесо офлайн по пропаже @F и останавливает робота
Потеря пакетов/битый CRC	Пакет отбрасывается, кадр синхронизируется по заголовку
Просадка АКБ	Двухпороговая политика: ползущий режим → аварийный стоп
Смерть ноды	Лаунчеры `respawn:=true`; миссия при FAULT останавливает робота через лимит 0
Нет признаков (лидар молчит)	`lds01_driver_node` публикует пустые сканы и статус, costmap считает лучи свободными только по свежим данным

---

9. Направления развития (не входит в текущую ревизию)

1. Замена классических CV-детекторов на YOLOv8-модель с весами, обученными на знаках полигона.
2. Фильтр локализации `robot_localization` (EKF), если появится IMU.
3. Behavior Tree (py_trees/BehaviorTree.CPP) вместо линейного FSM миссии.
4. Интерфейс оператора: веб-панель (телеметрия, телеуправление, карта).
5. Диагностика `diagnostic_msgs` по всем узлам.

---

10. Переход `module_link.py` на реальную прошивку

`rus_slam_base/module_link.py` — единственное место в ROS 2-стеке, которое трогает порт. Сейчас там `serial.Serial(...)` + `protocol.py` с бинарными кадрами. Чтобы стек заработал с fw26, нужно заменить три вещи, не меняя остальные узлы (топики `/wheel_commands`, `/modules/state`, `/battery` остаются):

Что	Сейчас	Должно быть (fw26)
Транспорт	10/16-байтные кадры, поиск `0xBB 0x44`	построчное чтение до `\n`, разбор `@F` / `@TLM`; в fw26 кадр `@F` — 22 слова, последнее перед CRC — измеренный угол ра
Команда руля	`STEER_CDEG` (i16)	`w <XX> <угол>` — угол в градусах, предел **±100°**, нужен ноль
Команда тяги	`TRACTION_PWM` −1000…+1000	`v <0…150> [f/b]` — прямое значение `Servo.write`, heartbeat чаще 650 мс
Включение	`FLAG_ENABLE`	отдельного флага нет: ненулевое v/t само разрешает выходы
Хоминг	`FLAG_HOME`	`w XX calib` (калибровка по опто) + `q` (потенциометр)
Сторож	300 мс на МК	**650 мс** на МК, плюс проверка свежести @F на сервере

Готовый разбор строк — в Zapret (`slam_gui/backend.py`: `parse_master_line`, `MasterSource._command_module`). Если переносить его в ROS 2, лучше вынести в общий модуль

и переиспользовать в обоих репозиториях, чтобы формат не разошёлся.

Проверка без железа: `firmware/firmware_sim/sim.cpp` в режиме `pty` `/tmp/wheel` поднимает четыре псевдотерминала с настоящей логикой прошивки — их можно подставить в `module_link.py` вместо `/dev/ttyUSB*`.